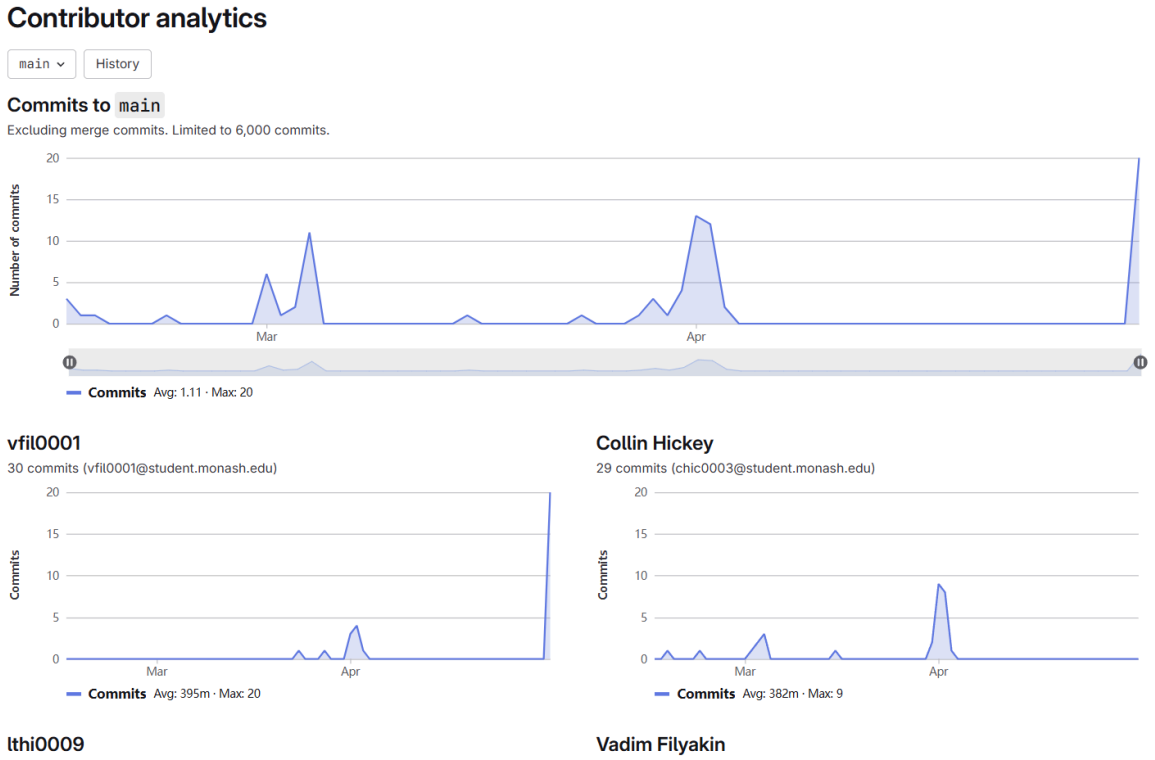
Vadim Filyakin 33764506

FIT3077 Software Engineering:

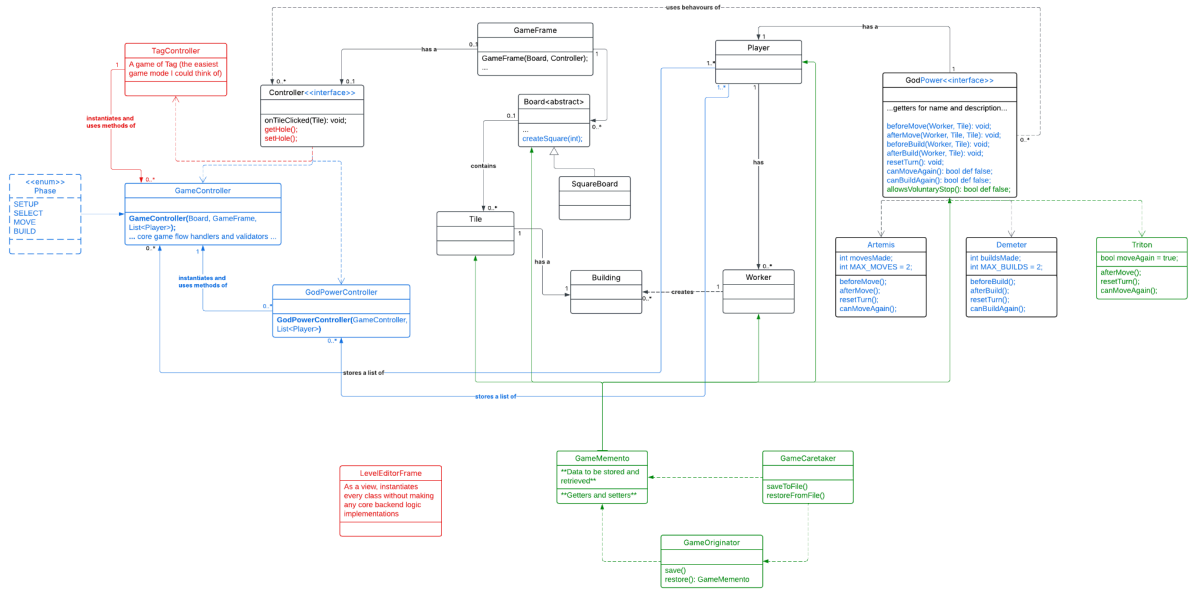
Architecture and Design

Sprint 3



Class Diagram

Vadim Filyakin 33764506



A brief description of the colour code used:

Red: added for the feature that is to be implemented during this spring (e.g. sprint 3).

Green: has been added for the current sprint (e.g. sprint 3), and partially updated after the feature has been implemented in case it may affect future feature implementations (track back to previous commits to see the whole history of the design)

Blue: a difference in the actual code for the previous sprint (e.g. sprint 2) and the hypothetical version of it that was created before the sprint (see justifications in the respective document in the sprint_two folder).

The history of the diagram can be tracked in git commits (repo—> Commits —> Ctr+F —> “Class Diagram”).

CRC Cards

GameMemento
Responsibility: 1. Combine and serialize the data of the current game state 2. Convert data from text (String) back into GameMemento
Collaborator: 1. Tile, Player, Worker, Controller, Board, GodPower

Originator
Responsibility: 1. Save the snapshot into a file 2. Retrieve the snapshot data from a file
Collaborator: GameMemento

Caretaker
Responsibility: 1. Hold the state of Originator 2. Control and bond Originator and Memento
Collaborator: Originator, GameMemento

Vadim Filyakin 33764506

Above are the CRC cards for the Task 2 classes. Both tasks implemented in Task 3 are either very similar to already implemented classes or completely separate from the main application logic (are classified as views).

Design Rationale

New interfaces and classes

- LevelEditorFrame is a very junky class responsible for the frontend of the level editor functionality. Technically, it could have been split into three different classes (one for each section of the grid), but that would significantly hinder loading time, considering it is already quite slow.
- GameFrame has been refactored almost ground up because the layering for that one used to be a nightmare to deal with. I could have left it the way it was, with everything, including the background image and the buttons being stacked up on the same overlay, but having to troubleshoot it from the get-go was very tedious and infuriating, even.
- GameMemento, GameOriginator, and GameCaretaker serve as examples of the Memento pattern. This approach is an excellent solution for managing save states or implementing an undo function. It effectively encapsulates all related logic within these three classes while minimising impact on the rest of the system. Although it's possible to combine GameOriginator and GameCaretaker into a single class, I believe it's generally better to separate responsibilities whenever possible
- Much like GodPowerController, TagController is an instance of a wrapper class for a decorator type of dynamic with GameController. It could have as well instanciated GodPowerController instead of GameController, but that would have made some sort of tree, and it’s generally bad to make trees out of things that should not be arranged that way logically.

Alternative Designs

I’ve been following MVC so far, and honestly, it’s been alright; the resistance from the system while implementing new stuff was insignificant for the most part. Besides, there are two things I would really love to refactor if only I had more time to do so:

1. Even though the Controllers are technically using a Template Method Pattern with the game phase switches, unfortunately, this design doesn’t accommodate enough leeway for me to turn the game of Santorini into a completely different game (like point capture or whatever) purely through the controllers. If given the opportunity, I would get rid of the enums and phases completely, and just hardcode/force them on a couple of standard controllers instead.
2. Throughout this sprint, I spent way too much time dealing with the resistance on the frontend of the system. Honestly, I refactored so much of it at this point, it makes me think that’s a miracle it worked during the previous sprint. Yet, I tried to save the core

of it (the root of all evil), and left the “modularity” of it intact, but I would definitely just redo the whole thing ground up in case we had another sprint.

Design Patterns used:

- **Memento Pattern**:

  Explanation: GameMemento, GameOriginator, and GameCaretaker enable the capture and restoration of the entire game state to and from disk.

  Reasoning: provides some great encapsulation to the feature; easy to implement.

- **Observer Pattern:**

  Explanation: Implemented through Swing. All the ActionListeners (and mouse listeners in the editor) act as observers. When the user clicks a tile or button, those listeners are notified and drive the appropriate controller methods.

  Reasoning: Had to do it one way or another to make some response to a dynamic input from the user.

- **Template Method Pattern:**

  Explanation: GameController defines the broad “phases” (SETUP, SELECT, MOVE, BUILD) and provides protected “hook” methods like handleMove() and handleBuild(). TagController (and GodPowerController by extension) override/extend those hooks to tweak only the parts they need, without having to rewrite the entire turn-cycling logic.

  Reasoning: The original idea was that it would make the implementation and design of new controllers more straightforward.

Human Value Extension:

Feature: Level Editor.
Human Value: Creativity (under Self Direction)

This feature provides the users with a comprehensive interface to make some brand new maps (boards), and even completely distinct game mods, and then consequently share this experience with other users by sharing their creations. The idea is that by enabling and even sort of encouraging the idea of users being able to play the game the way they want, and share this experience with friends and family, people are able to nurture some special bond. Honestly, I daresay, this type of flexibility is the actual reason why board games, as well as games in general, are so popular. Every household has its own UNO rules, and that makes it fun. When people share these unique experiences with others, it creates connections, and

these connections help to build a community. In my opinion, sharing the fruits of creativity is the essence of being creative.

In order to access the feature, users can either redirect to the Level Editor right from the starting menu or select a custom option for the mode field after the Start button is clicked. (For the sake of the assignment, I am providing CustomTagMod that technically utilises all the features from the editor.)

Vadim Filyakin 33764506

Sprint 2 Design Reflection

Task 1: Implementing a new GodPower

Honestly, I assumed from the very start that we would have to implement more God powers at one point or another. That is why I was prepared and intentionally made my design able to accommodate the implementation of any move or build-related God power. The interface has separate methods to define behaviour both before and after each phase, so implementing Triton was fairly easy. Besides, I had to add a new checker to the GodPower interface since Triton has a specific way to stop the movement of the player violation as long as they made at least one move. In hindsight, I think that adding new fields/ methods to an interface is a bad practice because, at some point, if I were to add 50 more god powers with specific requirements each, the interface would get very crowded and barely readable, which is also a sign of a code smell. Since I used a default value for it, I didn't even have to get into the code of the rest of the classes, so the implementation went very smoothly, I'd say.

Task 2: Save and Load

I have done the undo feature before for one of my personal projects, so I have some experience with memento. Besides, due to its modular nature, there are barely any cases where it is not fit for the job. Making the actual memento classes and testing them was very straightforward and easy. Perhaps me going through the guides on Refactoring.guru and Baeldung contributed to the success quite a bit, though. Besides, a thing that really threw me off was trying to add the save button to the GameFrame. Apparently, even though the preferred element dimensions were there, they were set to expand in both directions, given more room. Also, no grid was implemented. Therefore, whenever I tried to add a new button, the whole thing was freaking out. As a consequence of giving it an attempt to deal with, I spent about two hours trying to figure out a way to make it work and render the page correctly without me just redoing the whole frame code from scratch.

Task 3: Level Editor and TagController

TagController follows a decorator pattern with GameController, and it basically follows the design of the previously made GodPowerController. The LevelEditorFrame is a completely separate view-level class, it works similarly to GameFrame in essence, I attempted to make it more readable in practice. Besides, I had to add getter and setter methods for the holes in the Controller interface. Honestly, my initial design was to store the holes as boolean values in the Tile class, but halfway through implementing it, I realised that it would break my Save and Load functionality. Thus, I decided to make the controllers act as the access points to the tile info (so if the tiles store any value, like 1,1 or whatever, they return true; return false otherwise).